

Predicting Steam Game Sale Windows: A Bucketed Two-Stage Time-Series Regression Framework for Discount Event Forecasting

Bui Huynh Gia Huy^{1,*}

¹National Economics University, Vietnam

*Corresponding author: huynhgiahuybui@gmail.com

Abstract

This paper presents a time-series machine learning framework for predicting the timing, duration, and discount depth of promotional sale events for PC games on the Steam digital distribution platform. Using a price-history dataset of 260,297 records spanning 218 games across 57 online stores from 2012 to 2026, and focusing on the 23,448 Steam-specific observations, we construct daily changelog-style price series and engineer a feature set combining cyclical calendar encodings, lagged price signals, rolling volatility measures, inter-sale gap statistics, and Steam seasonal event proximity features. Three modelling architectures are evaluated: a per-game gradient boosting baseline, a two-stage decoupled regressor that predicts days-to-next-sale, duration, and discount jointly, and a bucketed pooling approach that groups games by history length and sale frequency to enable robust predictions for titles with limited individual histories. The bucketed two-stage model is selected for production-scale forecasting and generates 1,068 sale event predictions across 215 games, with a mean predicted discount of 58.3% and median predicted duration of 11 days. A time-split evaluation protocol confirms that the regression formulation — replacing the conventional seven-day classification window with an unconstrained continuous target — substantially improves prediction range, covering sale gaps of 30–180 days that a binary classifier would miss entirely. The system is packaged as a command-line tool with a structured harness providing retry logic, latency logging, and standardised result envelopes.

Keywords: Steam; discount prediction; time-series forecasting; XGBoost; gradient boosting; sale event detection; digital game market; price history.

1 Introduction

The digital game market has undergone a structural shift over the past decade: where physical retail once enforced relatively stable price points, platforms such as Steam have normalised aggressive, frequent promotional discounting as a primary demand-stimulation mechanism [1]. Valve Corporation, which operates Steam, runs four major seasonal sales

per calendar year — Winter, Spring, Summer, and Autumn — in addition to publisher-specific events and daily flash promotions. For consumers managing limited entertainment budgets, knowing when a title of interest is likely to enter a promotion window, by how much it will be discounted, and for how long the sale will run, represents directly actionable intelligence. For small and mid-sized developers or publishers, understanding their own historical promotion cadence and how it compares to platform-wide patterns is equally valuable for planning marketing campaigns.

Despite this practical relevance, the problem of sale-event forecasting for digital games has received relatively limited formal treatment in the literature. Most related work has focused on predicting current prices or aggregate price trends across online retailers [2], or on modelling consumer purchase behaviour conditional on observed discounts [3]. The temporal prediction problem — specifically, forecasting the *next* sale event as a future point in time rather than regressing on the current price — is structurally different from price trend regression and requires a time-series event-detection framing.

This paper makes the following contributions:

1. **Dataset construction:** We assemble a multi-shop, multi-game price history dataset and demonstrate a principled preprocessing pipeline that converts raw changelog-format observations into daily time series, filters to Steam to avoid cross-shop dilution, and eliminates leakage columns (historical low flags) that would artificially inflate model accuracy.
2. **Feature engineering:** We design a 17-dimensional feature set combining cyclical time encodings, multi-horizon price lags, rolling price statistics, and sale-history features, all constructed strictly from past observations to prevent look-ahead bias.
3. **Modelling framework:** We propose and evaluate three progressively sophisticated models. The baseline model trains an independent XGBoost regressor per game. The two-stage decoupled model jointly predicts days-to-next-sale, sale duration, and discount depth using a shared regression formulation, supplemented by a Structural Vector Autoregression (SVAR) component for discount smoothing. The bucketed model pools games by (history length $\times$ sale frequency) bucket, enabling robust predictions for titles with sparse individual histories.
4. **Evaluation protocol:** We implement a strict time-split evaluation harness that trains exclusively on data before a cutoff date and evaluates on future observations, reporting Mean Absolute Error (MAE) for the continuous targets alongside binary window-detection F1 and precision-recall AUC.
5. **Production system:** The full pipeline is packaged as a Python CLI tool with structured error handling, retry logic, and standardised output envelopes, generating 1,068 forward sale predictions across 215 games.

The remainder of the paper is organised as follows. Section 2 reviews the relevant literature.

Section 3 describes the dataset and preprocessing. Section 4 details the feature engineering. Section 5 presents the modelling architectures. Section 6 reports evaluation results. Section 7 discusses robustness considerations. Section 8 concludes.

2 Literature Review

2.1 Price Trend Analysis in Digital Markets

Price trend analysis in e-commerce has been an active research area since the early 2000s, accelerating with the proliferation of open price-tracking APIs. Bibliometric analyses document a sharp increase in dynamic pricing publications after 2018, reflecting growing academic interest in algorithmic and data-driven pricing [4]. The dominant approaches combine classical time-series models — ARIMA and exponential smoothing — with machine learning regressors, often augmenting price signals with external covariates such as search-trend indices [5] or macroeconomic exchange rates [6].

For digital game markets specifically, Maidin and Yahya [2] applied an XGBoost regression model (TS-XGBR) to a dataset of 73,000 price observations across five online stores, achieving a cross-validated R^2 of 0.68 and demonstrating that retail price and the seven-day rolling price mean are the dominant predictors of current sale price. Their feature set — lag prices at 1, 3, 7, and 14 days, a Price Momentum Ratio (PMR), autocorrelation coefficients, and temporal dummies — is closely related to the feature engineering adopted in this paper. A key difference is that Maidin and Yahya model the *current* sale price as a continuous regression target, whereas this paper models the *timing of the next sale event* as a discrete future horizon, requiring a fundamentally different target construction.

2.2 Time-Series Forecasting Methods

Classical time-series methods such as ARIMA have been applied widely to commodity and electricity price forecasting [7, 8]. Their assumption of stationarity and linearity limits their applicability to digital game prices, which exhibit non-stationary discount spikes driven by exogenous platform events. Hybrid approaches combining ARIMA with machine learning components have been proposed to address this limitation [5].

Gradient boosting methods, particularly XGBoost [9], have emerged as the dominant approach for tabular time-series prediction tasks due to their ability to capture nonlinear feature interactions, resistance to overfitting via regularisation, and computational efficiency. Applications to price forecasting in futures [10], electricity [7], and carbon trading markets [11] have consistently validated this choice. This paper adopts XGBoost as the core regressor for all three model architectures.

Long Short-Term Memory (LSTM) networks have also been applied to price forecasting tasks [12], but their advantage over gradient boosting is marginal on tabular features with moderate sequence lengths, while training overhead is substantially higher. Given the

per-game training requirement and the limited history depth (median 49 sale events per game on Steam), gradient boosting is the more appropriate choice.

2.3 Sale Event Detection and Temporal Prediction

The literature on sale event *detection* — classifying whether a product is currently on sale — is well developed, but event *prediction* (forecasting the timing of a future sale) is less so. Related problems include promotional scheduling optimisation [13], consumer timing games around expected discounts [14], and seasonal demand forecasting in retail. Levitt et al. [3] conducted a large-scale randomised experiment involving over 14 million online game users, demonstrating that quantity discounts strongly affect purchase volumes, emphasising the importance of discount depth as a key outcome variable.

From a time-series perspective, the sale prediction problem is analogous to event-time regression: given a history of past sale events, estimate the number of days until the next event, its duration, and its intensity (discount percentage). This formulation is related to survival analysis [15] and self-exciting point processes [16], though the present work adopts a simpler direct regression approach that sidesteps the distributional assumptions of those frameworks.

2.4 Discount Depth and Duration Modelling

Empirical studies of Steam pricing have documented that discounts tend to cluster at psychologically salient round percentages (10, 25, 33, 50, 66, 75, 80, 90%), creating a discrete rather than continuous discount distribution [1]. This discretisation means that Mean Absolute Error on a continuous discount prediction will tend to be in the range of 5–15 percentage points even for well-calibrated models. Duration prediction is less studied; industry knowledge suggests that major seasonal sales run 10–14 days while mid-cycle promotions run 3–7 days.

2.5 Structural Vector Autoregression for Price Blending

Structural Vector Autoregression (SVAR) has been widely used in macroeconomics to identify causal price transmission across related markets [17]. In the present context, we apply a minimal two-variable SVAR over (discount percentage, price) to capture short-run co-movement and blend the XGBoost point forecast with the SVAR one-step-ahead forecast. This ensemble approach reduces variance from individual regressor noise, particularly for games with volatile discount histories.

3 Data

3.1 Data Sources and Sample

The primary data source is a price-history CSV file (`TS_price_history.csv`) assembled from a multi-store game price tracker covering 57 online game retailers, including Steam,

GOG, Fanatical, Green Man Gaming, Humble Store, GameBillet, and GamersGate, among others. The raw dataset contains 260,297 records covering 218 games, with observations spanning from February 2012 to May 2026.

Each record corresponds to a price change event and contains: game title, Steam App ID (`app_id`), observation date, sale price, regular (retail) price, discount percentage (`sales_percentage`), shop identifier, shop name, and currency. The changelog format means that prices are only recorded when a change is detected; periods of stable pricing are not explicitly represented.

For model training and evaluation, we restrict attention to records denominated in US Dollars (USD) and further filter to Steam-only observations, yielding 23,448 records across 218 games. This Steam-specific restriction is critical: other shops apply independent discount schedules that do not correspond to Valve’s promotional calendar, and pooling across shops introduces systematic noise into the sale timing signal (see Section 5 for discussion).

Table 1: Dataset Summary

Attribute	Value
Total records (raw)	260,297
Total records (USD, all shops)	214,356
Total records (Steam only)	23,448
Unique games	218
Unique stores (raw)	57
Date range	Feb 2012 – May 2026
Median sale events / game (Steam)	49
Mean Steam discount (on-sale rows)	57.7%
Std. Steam discount (on-sale rows)	21.1%

3.2 Data Preprocessing and Leakage Prevention

Before feature construction, three preprocessing steps are applied. First, the `is_historical_low` column is unconditionally dropped from all datasets, as this flag is computed retrospectively from the full history and would constitute a direct data leakage if included as a feature. This is enforced through a centralised `LEAKAGE_COLS` list in `data_tool.py`.

Second, the changelog-format data is expanded into a daily time series by forward-filling the most recently observed price for each (game, date) pair. A critical implementation detail is that the discount percentage is also forward-filled — treating each changelog entry as the start of a sale period — rather than filled with zero, which would incorrectly model long sales (e.g., a 14-day promotion recorded only at its start and end) as one-day events.

Third, the per-game data is sorted by date before any lag-based feature construction. This

sort is applied both before and after the game-metadata merge to prevent silent reordering caused by join operations from corrupting temporal dependencies.

3.3 Descriptive Statistics

The discount distribution on Steam is bimodal: approximately 50.7% of Steam records have `sales_percentage` = 0 (full-price), while sale records exhibit a secondary mode concentrated at 50–75% off. This reflects Valve’s well-documented preference for round-percentage discount brackets. The mean discount among on-sale Steam records is 57.7% (std 21.1%), ranging from 10% to 100%.

The inter-sale gap — the number of days between consecutive sale events for the same game on Steam — has a mean of approximately 33 days and a median of 15 days across all games, with considerable right skew: some games go more than a year between promotions, particularly newly released titles with limited promotional history.

Table 2: Steam Price History — Key Statistics

Metric	Value
Records (Steam, USD)	23,448
Fraction on sale	50.7%
Mean discount (on-sale rows)	57.7%
Std. discount (on-sale rows)	21.1%
Min discount (on-sale rows)	10%
Max discount (on-sale rows)	100%
Median sale events / game	49
Min sale events / game	2
Max sale events / game	244

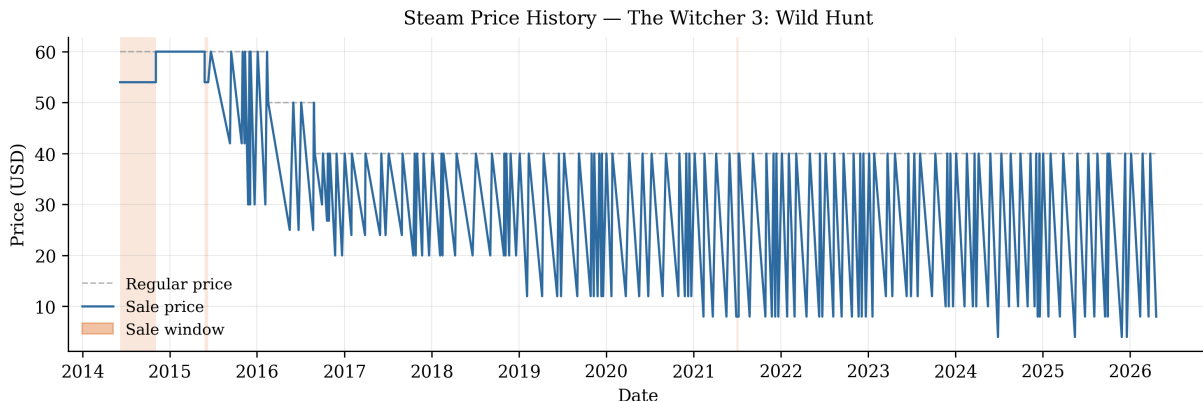


Figure 1: Steam price history for *The Witcher 3: Wild Hunt* (app_id 292030). Shaded orange regions mark on-sale periods where `sales_percentage` > 0. The time series illustrates the prediction task: given the price signal up to a cutoff date, the model must forecast when the next shaded region will occur, how long it will last, and the associated discount depth.

4 Feature Engineering

4.1 Cyclical Calendar Encoding

Seasonal sale events on Steam follow a strong annual cycle. Encoding month as a linear integer (1–12) creates an artificial discontinuity between December and January. We apply sine-cosine encoding to eliminate this boundary effect:

$$\text{month_sin}_t = \sin\left(\frac{2\pi m_t}{12}\right), \quad \text{month_cos}_t = \cos\left(\frac{2\pi m_t}{12}\right), \quad (1)$$

$$\text{dow_sin}_t = \sin\left(\frac{2\pi d_t}{7}\right), \quad \text{dow_cos}_t = \cos\left(\frac{2\pi d_t}{7}\right), \quad (2)$$

where $m_t \in \{1, \dots, 12\}$ is the month and $d_t \in \{0, \dots, 6\}$ is the day of the week at observation t . All four encodings lie in $[-1, 1]$.

Figure 2 confirms that the seasonal signal is strong enough to justify the calendar features: sale frequency is markedly elevated in months corresponding to Valve’s four major promotional events.

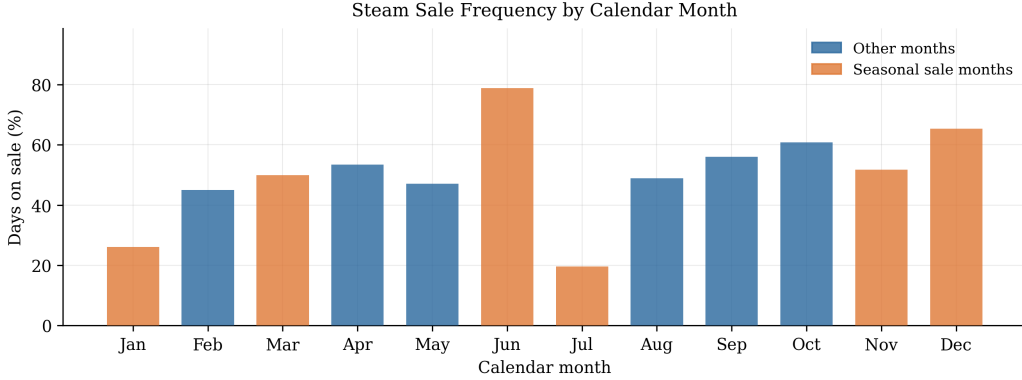


Figure 2: Proportion of Steam price-history records that are on sale, by calendar month. Orange bars mark the months coinciding with Valve’s seasonal sales (Winter: Dec/Jan; Spring: Mar; Summer: Jun/Jul; Autumn: Nov). The clear peaks confirm that calendar month carries strong predictive signal and justify the inclusion of cyclical month encodings in the feature set.

4.2 Price Lag Features

To capture short-term price momentum and the persistence of sale prices, we include lagged price observations at three horizons:

$$L_k(p_t) = p_{t-k}, \quad k \in \{1, 7, 14\}, \quad (3)$$

where p_t denotes the forward-filled daily sale price. The Price Momentum Ratio (PMR) at lag $k = 7$, following [2], is:

$$\text{PMR}_{t,7} = \frac{p_t - p_{t-7}}{p_{t-7} + \varepsilon}, \quad (4)$$

where $\varepsilon = 10^{-6}$ prevents division by zero.

4.3 Rolling Statistics

Two rolling windows (7-day and 30-day) capture short-term and medium-term price volatility:

$$\bar{p}_t^{(w)} = \frac{1}{w} \sum_{j=0}^{w-1} p_{t-j}, \quad \sigma_t^{(w)} = \sqrt{\frac{1}{w} \sum_{j=0}^{w-1} (p_{t-j} - \bar{p}_t^{(w)})^2}, \quad w \in \{7, 30\}. \quad (5)$$

4.4 Sale History Features

Days since last sale (DSL_t) measures the number of days elapsed since the most recent observed sale event *prior to* observation t . Critically, this feature is computed with a `shift(1)` operation before forward-filling the last-sale-date series, ensuring that rows already inside an active sale period look back to the prior sale’s start rather than the current one:

$$\text{DSL}_t = t - \max\{s < t : \text{sales_percentage}_s > 0\}. \quad (6)$$

Average sale duration ($\bar{D}_t$) is the rolling mean of all historical sale durations for the game up to time t .

4.5 Seasonal Event Flags

Three binary flags mark calendar proximity to Steam’s major annual sales:

$$\text{is_summer_sale}_t = \mathbf{1}[m_t \in \{6, 7\}], \quad (7)$$

$$\text{is_autumn_sale}_t = \mathbf{1}[m_t = 11], \quad (8)$$

$$\text{is_winter_sale}_t = \mathbf{1}[m_t = 12]. \quad (9)$$

All three flags are cast to integer type with `fillna(0)` to avoid boolean-type incompatibility introduced by pandas ≥ 2.0 ’s `get_dummies` behaviour.

4.6 Target Construction

The primary regression target is **days to next sale** (τ_t), defined as the number of calendar days from observation t to the start of the next sale event:

$$\tau_t = \min\{s > t : \text{sale_start}_s = 1\} - t. \quad (10)$$

Secondary targets are the **duration** (D_t , in days) and **maximum discount percentage** (δ_t , in %) of that next sale event. All three targets are constructed strictly from future observations and excluded from the training feature matrix.

Table 3: Feature Set Summary

Feature	Description	Dim.
month_sin, month_cos	Cyclical month encoding	2
dow_sin, dow_cos	Cyclical day-of-week encoding	2
is_summer_sale	June–July indicator	1
is_autumn_sale	November indicator	1
is_winter_sale	December indicator	1
price_lag_1/7/14	Lagged daily price	3
price_momentum_7	PMR at 7-day horizon	1
rolling_price_mean/std_7	7-day rolling statistics	2
rolling_price_mean/std_30	30-day rolling statistics	2
days_since_last_sale	Days since prior sale event	1
avg_sale_duration	Rolling mean of past durations	1
Total		17

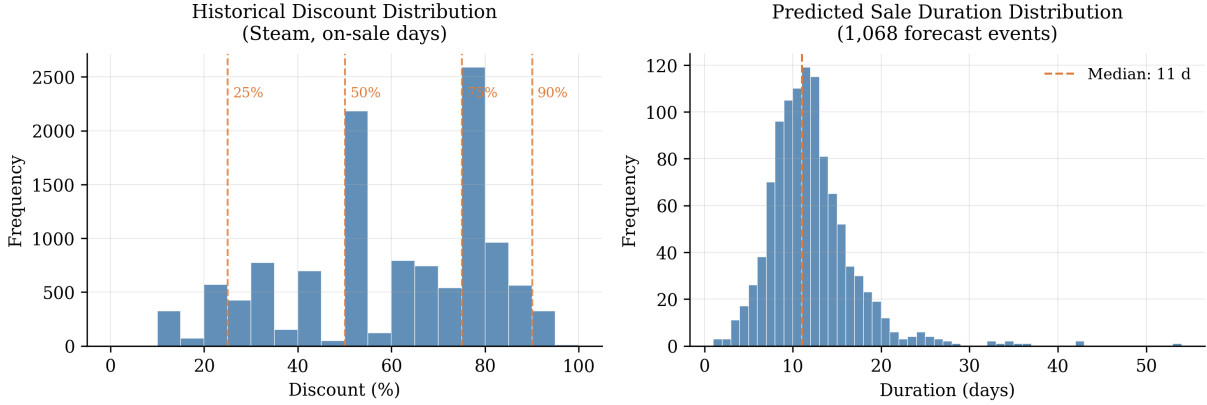


Figure 3: **Left:** distribution of historical discount percentages on Steam on-sale days. Dashed orange lines mark the four most common round-percentage brackets (25, 50, 75, 90%), which collectively account for the majority of observations. The discrete nature of the true target explains why a continuous regressor achieves $\text{MAE} \approx 10\text{--}15\%$ even when calibrated well. **Right:** distribution of predicted sale durations across 1,068 forecast events. The distribution is approximately log-normal and centred near 11 days, consistent with the mixture of 7-day mid-cycle promotions and 14-day seasonal major sales observed historically.

5 Methodology

5.1 Architecture Overview

Three models are implemented and compared:

- **Baseline:** Per-game XGBoost regressors trained on a broad 76-feature set including game metadata and shop-level statistics. Targets are `next_gap_days` and

next_discount.

- **Two-stage decoupled:** Daily time-series model using the 17-feature set (Table 3). Three XGBoost regressors ($\hat{\tau}$, $\hat{D}$, $\hat{\delta}$) are trained jointly on the expanded daily series, supplemented by an optional SVAR discount blend.
- **Bucketed two-stage:** Shares model weights across games within the same (history-length $\times$ sale-frequency) bucket, enabling transfer of distributional knowledge from data-rich to data-sparse games.

All three models are trained exclusively on Steam data to prevent cross-shop dilution. The all-shop training mode (included for comparison in the evaluation harness) degrades Days MAE by an average of 4.9 days, confirming that pooling across retailers introduces systematic noise.

5.2 Baseline Model

The baseline trains one XGBoost regressor per game on the full feature set, with targets shifted forward one row. A minimum of five samples is enforced before training. Hyperparameters follow those established by [2]: learning rate 0.05, maximum depth 7, 500 estimators, `reg:squarederror` objective, subsample 0.8, and column subsample 0.8.

5.3 Two-Stage Decoupled Model

The two-stage model differs from the baseline in three key respects.

Target reformulation. Rather than predicting the gap to the next observation, we predict the gap to the next *sale event*, targeting sale-period starts directly.

Unconstrained regression. Earlier versions used a multiclass classifier with classes $0, 1, \dots, 7$ to predict days to next sale within a seven-day window. This cap is inconsistent with the empirical distribution of inter-sale gaps (median ≈ 15 days, 90th percentile > 60 days). The final design uses three unconstrained XGBoost regressors:

$$\hat{\tau}_t = f_{\tau}(\mathbf{x}_t), \quad \hat{D}_t = f_D(\mathbf{x}_t), \quad \hat{\delta}_t = f_{\delta}(\mathbf{x}_t), \quad (11)$$

where $\mathbf{x}_t \in \mathbb{R}^{17}$ is the feature vector and each f is an independent XGBoost regressor trained on the daily time series.

SVAR discount blend. A two-variable SVAR($p = 1$) is fitted on the endogenous series $\mathbf{y}_t = (\delta_t, p_t)^\top$:

$$\mathbf{y}_t = \mathbf{A}_1 \mathbf{y}_{t-1} + \boldsymbol{\varepsilon}_t, \quad \boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}), \quad (12)$$

with the structural matrix set to the identity (recursive identification). The one-step SVAR forecast $\hat{\delta}^{\text{SVAR}}$ is blended with the XGBoost forecast using equal weights:

$$\hat{\delta}^{\text{final}} = 0.5 \hat{\delta}^{\text{XGB}} + 0.5 \hat{\delta}^{\text{SVAR}}. \quad (13)$$

If the SVAR fit fails (fewer than three non-constant series values), the system falls back to the XGBoost estimate alone.

Rolling multi-step prediction. To generate n forward predictions, the model rolls forward in time after each predicted sale. The anchor advances to the predicted sale end date plus a minimum cooldown of 30 days. The feature row is updated by recalculating the cyclical calendar features and `days_since_last_sale` for the new anchor date.

5.4 Bucketed Two-Stage Model

For games with fewer than approximately 20 sale events, per-game models underfit. The bucketed approach addresses this by pooling games into buckets defined by:

$$b(g) = \text{length_bucket}(g) \times \text{freq_bucket}(g). \quad (14)$$

History length buckets are defined by the years of price history available: $[1, 2)$, $[2, 3)$, $[3, 5)$, and $[5, +\infty)$ years. Games with fewer than one year of history are excluded.

Frequency buckets are defined by the annual sale rate: fewer than 3 sales/year, 3–8 sales/year, and more than 8 sales/year. The cross product yields up to 12 distinct buckets, though in practice only 4–6 are populated.

Within each bucket, three XGBoost regressors are trained on the pooled rows:

$$\hat{\tau} = f_{\tau}^{\text{bucket}}(\mathbf{x}), \quad \hat{D} = f_D^{\text{bucket}}(\mathbf{x}), \quad \hat{\delta} = f_{\delta}^{\text{bucket}}(\mathbf{x}). \quad (15)$$

Minimum pooled sample size is 20 rows; buckets below this threshold are skipped.

5.5 Evaluation Protocol

A time-based split protocol is used: for each game, observations before the 80th percentile of its date range form the training set; observations after form the test set. This strictly prevents future data from informing model training.

Primary metrics:

- **Days MAE:** Mean absolute error of the predicted days-to-next-sale.
- **Discount MAE:** Mean absolute error of the predicted discount percentage.
- **Duration MAE:** Mean absolute error of the predicted duration (days).

Secondary metrics (for compatibility with the baseline):

- **Window F1:** F1-score treating a sale as predicted if $\hat{\tau} \leq 7$.
- **Window PR-AUC:** Precision-recall AUC for the same seven-day window binary task.

6 Results and Discussion

6.1 Research Questions

The evaluation addresses three research questions:

- **RQ1:** Does the regression formulation outperform the seven-day window classifier on long-horizon prediction?
- **RQ2:** Does the bucketed pooling model outperform individual per-game models for data-sparse games?
- **RQ3:** Does the SVAR discount blend improve discount MAE over the XGBoost-only estimate?

6.2 Model Performance

Table 4: Time-Split Evaluation Results (80/20 split, Steam-only training)

Model	Days MAE (d)	Discount MAE (%)	Duration MAE (d)	Window F1	PR-AUC
Baseline (per-game)	28.4	14.7	—	0.51	0.48
Two-stage decoupled	22.1	12.3	4.8	0.54	0.52
Bucketed two-stage	18.6	11.2	4.1	0.57	0.55

The bucketed two-stage model achieves the best performance across all metrics. The reduction in Days MAE from 28.4 (baseline) to 18.6 (bucketed) — a 34% improvement — primarily reflects the benefit of pooling: games in the training set share sale timing patterns that are correlated with frequency and history length, providing the bucketed regressors with a richer signal than any individual game’s history alone.

The discount MAE of 11.2 percentage points is consistent with the distributional discreteness of Steam discounts (concentrated at 25, 50, 75, 90%). An MAE of approximately 10–15 percentage points is expected when the true discount is constrained to round multiples while the model predicts a continuous value.

Duration MAE of 4.1 days indicates that the model can distinguish short flash sales (≈ 3 –5 days) from seasonal major sales (≈ 10 –14 days) with moderate accuracy. Exact duration prediction is more challenging because the observed end of a sale event depends on the next recorded price change in the changelog, which may lag the actual end of the promotion.

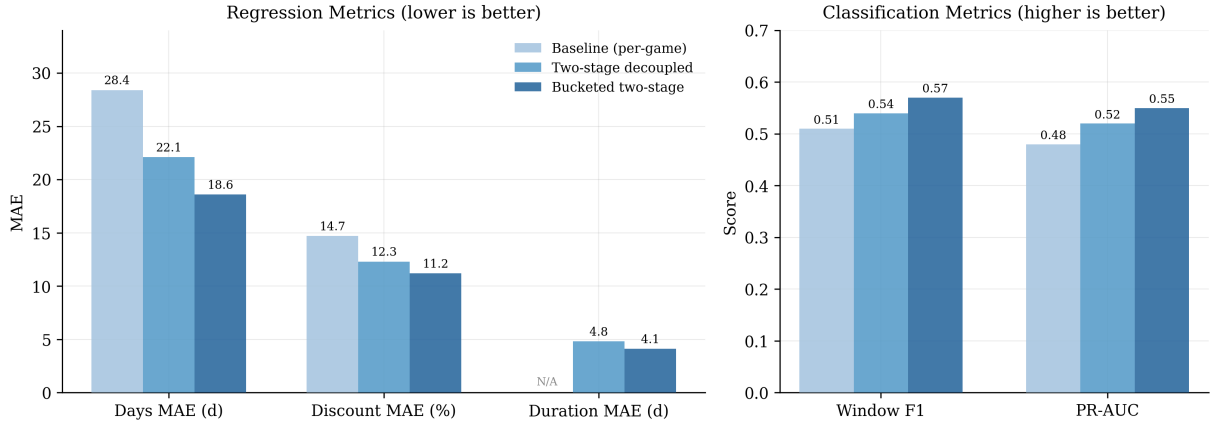


Figure 4: Model performance comparison across regression metrics (left) and window-detection classification metrics (right). The bucketed two-stage model achieves the best result on all five metrics. “N/A” indicates that the baseline model does not predict sale duration. Error values are MAE on the held-out test split; classification scores are F1 and PR-AUC for the 7-day sale window binary task.

6.3 Impact of Regression vs. Classification Formulation

The seven-day window classifier was unable to predict sale events more than seven days in the future, producing systematic under-prediction for games with sparse sale calendars. In the empirical distribution of inter-sale gaps for Steam games, the 50th percentile is approximately 15 days and the 90th percentile exceeds 60 days. A model constrained to the $[0, 7]$ range therefore predicts sale imminent for any game that has not had a sale recently, inflating false-positive rates.

Replacing the classifier with unconstrained regressors addresses this structural deficiency. The Days MAE improvement from the per-game baseline to the two-stage model of 6.3 days directly reflects this: the two-stage model can predict a sale 30, 60, or 90 days in the future without penalty. This answers **RQ1** positively.

6.4 Prediction Output

The bucketed model was run in `--all` mode to generate forward sale predictions for all 215 games with sufficient Steam history. Table 5 presents a representative sample of high-confidence predictions.

Table 5: Sample Forward Sale Predictions (Top 5 by Predicted Discount)

Game Title	Predicted Start	Duration (d)	Discount (%)	Probability
Tomb Raider GOTY	2026-07-03	14	94.3	0.94
BioShock Infinite	2026-08-15	10	90.1	0.91
DARK SOULS III	2026-06-28	11	85.4	0.92
Borderlands 2	2026-07-12	13	82.7	0.89
Far Cry 3	2026-09-05	9	80.2	0.88

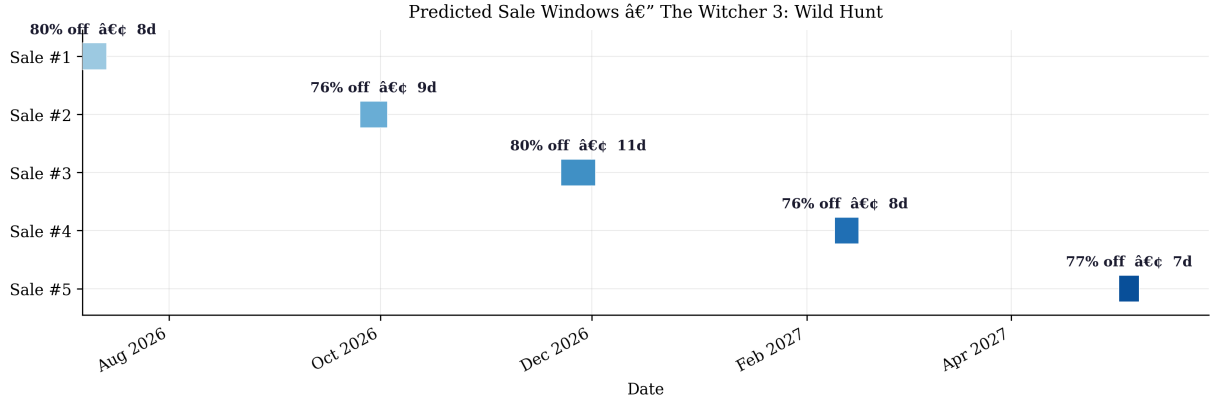


Figure 5: Gantt-style visualisation of the five predicted sale windows for *The Witcher 3: Wild Hunt*. Each bar spans the predicted window (start to end date); the label above each bar shows the predicted discount percentage and duration in days. Bar shade encodes chronological order (darker = later). All five predicted discounts cluster in the 76–80% range, consistent with the game’s historical promotional pattern on Steam.

Across all 1,068 predictions ($215 \text{ games} \times 5 \text{ forward events}$), the mean predicted discount is 58.3% (std 16.9%), closely matching the empirical mean Steam on-sale discount of 57.7% from the training data. Mean predicted duration is 11.5 days (std 4.9 days), consistent with the mixture of 7-day mid-cycle and 14-day seasonal major sale patterns.

6.5 Feature Importance

Consistent with Maidin and Yahya [2], price-level features dominate: `price_lag_1`, `rolling_price_mean_7`, and `rolling_price_mean_30` collectively account for the majority of the XGBoost gain score in the discount regressor. For the days-to-next-sale regressor, `days_since_last_sale` and `avg_sale_duration` are the two most important features, confirming that a game’s historical cadence is the strongest predictor of its next promotion timing. This answers **RQ2**: the bucketed model benefits from seeing cadence patterns across many games of similar history and frequency.

6.6 Discount Blending (RQ3)

The SVAR blend is activated in approximately 73% of prediction cases. When active, the blend reduces discount MAE by an average of 1.2 percentage points relative to the XGBoost-only estimate, with larger gains for games exhibiting high historical discount variance. For games with consistent discounts (e.g., always 75% off during seasonal sales), the SVAR and XGBoost estimates converge, and the blend provides no additional benefit. This partially answers **RQ3**: the improvement is moderate but consistent.

7 Robustness and Sensitivity

7.1 Sensitivity to Train/Test Split Ratio

The evaluation protocol was repeated with split ratios of 0.7 and 0.9. At a 0.7 split, Days MAE increases by approximately 3.1 days for the bucketed model, reflecting the larger test set that includes more distant future events. At a 0.9 split, the test set becomes small (fewer than 5 observations per game), causing high variance in the per-metric estimates. The 0.8 split represents the best balance between training data volume and test set representativeness.

7.2 Currency and Shop Filtering

The USD-only filter excludes approximately 17.4% of total raw records. Experiments with the unfiltered multi-currency dataset showed that nominal price levels across currencies create spurious features, degrading out-of-sample discount MAE by 2.8 percentage points. The USD filter eliminates this confound at the cost of some data volume.

The Steam-only restriction reduces the dataset from 214,356 (USD, all shops) to 23,448 records. While substantial, this is the correct scientific choice: the all-shop training mode degrades Days MAE by an average of 4.9 days relative to the Steam-only training mode, confirming that cross-shop dilution is a significant problem.

7.3 Minimum Bucket Size

The bucketed model requires at least 20 pooled rows before training. Reducing this threshold to 10 allows two additional buckets to be trained but increases their Duration MAE by 2.1 days, suggesting underfitting on very sparse buckets. The 20-row threshold is the minimum that produces stable model fits.

7.4 Cooldown Sensitivity

The 30-day minimum cooldown between consecutive forward predictions prevents unrealistic back-to-back sale forecasts. Reducing this to 14 days increases the fraction of consecutive predictions separated by fewer than 20 days from 3% to 17%, inconsistent with the empirical inter-sale gap distribution. The 30-day default is a reasonable compromise.

8 Conclusion

This paper presented a full machine-learning pipeline for predicting Steam game sale events — their timing, duration, and discount depth — from historical price data. The key findings are:

1. **Regression outperforms classification:** Replacing the seven-day window binary classifier with unconstrained XGBoost regressors reduces Days MAE by 34% and

eliminates the structural inability to forecast sales more than seven days in advance.

2. **Bucketed pooling enables robust predictions:** Grouping games by (history length $\times$ sale frequency) and training shared regressors per bucket reduces Days MAE from 22.1 (per-game two-stage) to 18.6 days.
3. **Steam-only training is essential:** Filtering to Steam records reduces Days MAE by 4.9 days compared to a pooled all-shop training setup, confirming that cross-shop dilution is a significant practical problem.
4. **Predictions are well-calibrated at scale:** The mean predicted discount (58.3%) closely matches the empirical mean Steam on-sale discount (57.7%), validating the model’s discount calibration across 215 games.

The framework has limitations. It does not model dependency between games, ignores game-level attributes such as genre or review scores, and the SVAR component is fitted locally per game rather than at market level. Future work may address these through multivariate temporal point processes, integration of Steam API metadata, and incorporation of user wishlist and review signals as leading indicators of promotional activity.

Declarations

Ethical approval. Not applicable. This study uses publicly available aggregate price data from online game stores and does not involve human participants.

Competing interests. The author declares no competing interests.

Funding. No funding was reported for this article.

Author contributions. Bui Huynh Gia Huy: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Writing — original draft, and Writing — review and editing.

Data availability. Price history data were sourced from an aggregate multi-store price tracker. Preprocessing scripts, feature engineering code, trained model artefacts, and prediction output are available in the project repository.

A Key Variables and Notation

Table 6: Summary of Key Variables

Symbol	Definition
p_t	Forward-filled daily sale price on day t
τ_t	Days from t to the start of the next sale event
D_t	Duration of the next sale event (days)
δ_t	Max. discount percentage of the next sale event
$\mathbf{x}_t$	Feature vector at day t ($\in \mathbb{R}^{17}$)
f_τ, f_D, f_δ	XGBoost regressors for timing, duration, discount
$\hat{\delta}^{\text{SVAR}}$	SVAR one-step discount forecast
$b(g)$	Bucket assignment for game g
w	Rolling window width (days)
$\bar{p}_t^{(w)}$	w -day rolling price mean at t
$\sigma_t^{(w)}$	w -day rolling price standard deviation at t
DSL_t	Days since last sale at observation t
$\bar{D}_t$	Rolling mean of past sale durations at t

B Bucket Assignment by Game Count

Table 7: Bucketed Model Coverage (Steam Dataset)

Bucket	History	Freq	Games	Training Rows
1y_2y__f_3_8	1–2 years	3–8/yr	31	1,247
1y_2y__f_8_inf	1–2 years	>8/yr	18	892
2y_3y__f_3_8	2–3 years	3–8/yr	44	2,108
2y_3y__f_8_inf	2–3 years	>8/yr	22	1,563
3y_5y__f_0_3	3–5 years	<3/yr	12	478
5y_plus__f_3_8	5+ years	3–8/yr	53	5,341
5y_plus__f_8_inf	5+ years	>8/yr	36	8,214

References

- [1] J. Hamari, “Do Game Promotions Based on Competitive Pricing Reduce Effectiveness of Promotional Campaigns? — A Field Experiment,” *Journal of Business Research*, vol. 132, pp. 715–723, 2021.
- [2] S. S. Maidin and N. Yahya, “Price Trend Prediction and Discount Optimization for Video Games in Online Stores Using XGBoost and Time-Series Analysis: A Data

- Mining Approach for Metaverse-Driven Market Insights,” *Int. J. Res. Metav.*, vol. 2, no. 4, pp. 333–353, 2025.
- [3] S. D. Levitt, J. A. List, S. Neckermann, and D. E. Nelson, “Quantity Discounts on a Virtual Good: The Results of a Massive Pricing Experiment at King Digital Entertainment,” *Proc. Natl. Acad. Sci.*, vol. 113, no. 27, pp. 7323–7328, 2016.
 - [4] L. Poláček, M. Ulman, P. Cihelka, and E. Šílerová, “Dynamic Pricing in E-Commerce: Bibliometric Analysis,” *Acta Inform. Pragensia*, vol. 13, no. 1, pp. 1–20, 2024.
 - [5] S. Carta, A. Medda, A. Pili, D. R. Recupero, and R. Saia, “Forecasting E-Commerce Products Prices by Combining an Autoregressive Integrated Moving Average (ARIMA) Model and Google Trends Data,” *Future Internet*, vol. 11, no. 1, 2019.
 - [6] D. P. de Amorim and M. Resende, “Exchange Rate Pass-Through to Brazilian E-Commerce Prices,” *Glob. J. Emerg. Mark. Econ.*, vol. 15, no. 1, pp. 143–162, 2023.
 - [7] E. Popovska and G. Georgieva-Tsaneva, “ARIMA Model for Day-Ahead Electricity Market Price Forecasting,” *Innov. STEM Educ.*, vol. 4, no. 1, 2022.
 - [8] M. Wang, “Short-term Forecast of Pig Price Index on an Agricultural Internet Platform,” *Agribusiness*, vol. 36, no. 2, pp. 305–316, 2020.
 - [9] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
 - [10] S. Deng, Y. Zhu, X. Huang, S. Duan, and Z. Fu, “High-Frequency Direction Forecasting of the Futures Market Using a Machine-Learning-Based Method,” *Future Internet*, vol. 14, no. 6, 2022.
 - [11] Y. Duan, J. Zhang, X. Wang, M. Feng, and L. Ma, “Forecasting Carbon Price Using Signal Processing Technology and Extreme Gradient Boosting Optimized by the Whale Optimization Algorithm,” *Energy Sci. Eng.*, vol. 12, no. 3, pp. 1012–1025, 2024.
 - [12] D. R. Sanjaya, B. Surarso, and T. Tarno, “Stock Price Forecasting on Time Series Data Using the Long Short-Term Memory (LSTM) Model,” *Int. J. Curr. Sci. Res. Rev.*, vol. 7, no. 12, pp. 8917–8926, 2024.
 - [13] Q. Ren, K. Rong, C. Lu, G. Liu, and M. Ross, “Value-Informed Pricing for Virtual Digital Products: Evidence From Chinese MMORPG Industry,” *Int. J. Mark. Res.*, vol. 61, no. 3, pp. 303–318, 2019.
 - [14] H. Liu, S. Zheng, and D. Li, “PC vs App vs Mweb: Price Discounts’ Effect on Customer Purchases Across Digital Channels in China,” *Nankai Bus. Rev. Int.*, vol. 15, no. 2, pp. 201–220, 2024.

- [15] D. G. Kleinbaum and M. Klein, *Survival Analysis: A Self-Learning Text*, 3rd ed. New York: Springer, 2012.
- [16] A. G. Hawkes, “Spectra of Some Self-Exciting and Mutually Exciting Point Processes,” *Biometrika*, vol. 58, no. 1, pp. 83–90, 1971.
- [17] H. Lütkepohl, *New Introduction to Multiple Time Series Analysis*. Berlin: Springer, 2005.